MSH 3-T-junction array: logical gates

Function reference

`logical_gates_tn.ipynb`	Julia: reduced method from the tensor-network (TDVP) evolution
`logical_gates_python_ed.ipynb`	Python: full Pfaffian method and reduced method, dense
`tjunctions_model.py`	Python: shared model of the array (with `tjunctions_python_ed.ipynb`)

Repository `MSH_Tensor_Network_Methods`, branch `tjunctions-logical-gates`

30 October 2026

This document explains every function defined in the two logical-gate notebooks, and the functions of `tjunctions_model.py` they use: what each computes, the method, inputs and outputs, and the numerical settings that matter. All numbers are quoted from the saved outputs of the executed notebooks. The model itself (geometry, Hamiltonian, protocols, zero-mode construction) is described in detail in `docs/msh_tjunctions_function_reference.pdf`; here it is only summarized.

Contents

1 Overview **3**

1.1 What is computed . . . 3

1.2 Workflow and inputs . . . 3

1.3 Results . . . 4

2 Method **4**

2.1 Quasiparticle operators and the Majorana matrix . . . 4

2.2 Spin representation and the Jordan–Wigner convention . . . 4

2.3 Full method: generalized Wick theorem . . . 4

3 Part I — `logical_gates_tn.ipynb` (Julia) **5**

3.1 Section 1: setup and the cached overlaps . . . 5

3.2 Section 2: from the Majorana matrix to the logical gate . . . 5

3.3 Section 3: logical gates after one and two exchanges . . . 6

3.4 Section 4: export . . . 6

4 Part II — `logical_gates_python_ed.ipynb` (Python) **6**

4.1 Code from `tjunctions_model.py` . . . 6

4.2 Section 1: setup . . . 7

4.3 Section 2: time evolution of the full frame . . . 7

4.4 Section 3: full method . . . 7

4.5 Section 4: reduced method . . . 8

4.6 Section 5: full vs reduced 8
4.7 Section 6: comparison with the tensor-network gates 9

5 `tjunctions_model.py` 9

1 Overview

1.1 What is computed

The array of `msh_tjunctions_tn.ipynb` carries four Majorana zero modes $\gamma_1, \ldots, \gamma_4$, paired into the positive-energy fermion modes $\psi_A = (\gamma_1 + i\gamma_2)/\sqrt{2}$ (junction 1) and $\psi_B = (\gamma_3 + i\gamma_4)/\sqrt{2}$ (junction 3). The logical qubit is the even-parity pair

$$|0\rangle_L = |00\rangle, \qquad |1\rangle_L = |11\rangle,$$

where $|n_A n_B\rangle$ has mode ψ_m occupied when $n_m = 1$ (the convention of the masks in the CZ, QFT and period-finding notebooks). The two gate protocols of the T-junction notebooks are analysed after one and after two exchanges:

Protocol	Exchanges	T_1	One exchange	Two
`exchange_12` (cell 1, `rot_Tz = 200`)	$\gamma_1 \leftrightarrow \gamma_2$ (same fermion)	2040	$\sqrt{Z}^\dagger$	Z
`exchange_23` (cell 2, `rot_Tx = 400`)	$\gamma_2 \leftrightarrow \gamma_3$ (different fermions)	3880	$\sqrt{X}^\dagger$	X

The result for each is the **logical unitary** on $\{|0\rangle_L, |1\rangle_L\}$, with signs and relative phases, by two methods:

- **Full method** (`signed_amplitudes`): the many-body amplitudes $\langle j|\hat{U}|k\rangle$ of the Gaussian (BCS) states from the single-particle propagator S_T applied to all $2N$ occupied quasiparticles, via the generalized Wick theorem (a Pfaffian). Gives in addition the probability that the bulk stays unexcited. Needs the dense $4N \times 4N$ propagator.
- **Reduced method** (`braid_matrix`, `logical_unitary`): only the zero-mode block, the real 4×4 Majorana matrix $M_{ij} = \langle\gamma_i(0)|S_T|\gamma_j(0)\rangle$ and its spin representation. Needs only the two evolved mode vectors, so it works with MPS (and with sparse propagators); it cannot see bulk excitations.

1.2 Workflow and inputs

`msh_tjunctions_tn.ipynb` (TDVP caches) → `logical_gates_tn.ipynb` → `logical_gates_tn.csv` → `logical_gates_python_ed.ipynb`

File	Role
`msh_tjunctions_tdvp_{Z,X}.jls`	TDVP results of `msh_tjunctions_tn.ipynb` (section 6), including the overlaps $z_{ik}(t) = \langle\gamma_i(0)|\psi_k(t)\rangle$ at every snapshot. Not committed (`*.jls`); about 2.5 h of TDVP with six threads to create.
`logical_gates_tn.csv`	M and the leakage per gate and stage, from the TN notebook; committed; read by the Python notebook.
`logical_gates_python_ed_data/frames.npz`	initial frame ψ_0 and the evolved frames after one and two exchanges; not committed; recomputed when missing or with `RUN_EVOLUTION = True`.

1.3 Results

Gate	$P(00 \to 11)$	θ	best ideal gate (overlap)	unexcited prob.	full vs reduced	$\max\lvert\lvert M_{\mathrm{TN}}\rvert - \lvert M_{\mathrm{ED}}\rvert\rvert$
$\sqrt{Z}^\dagger$ (one `exchange_12`)	0	90.7180°	$\sqrt{Z}^\dagger$ (0.999980)	0.999998	3.3×10^{-12}	4.9×10^{-9}
Z (two)	0	178.5641°	Z (0.999921)	0.999997	1.0×10^{-11}	8.5×10^{-9}
$\sqrt{X}^\dagger$ (one `exchange_23`)	0.522078	92.5308°	$\sqrt{X}^\dagger$ (0.999756)	0.999982	9.8×10^{-8}	1.6×10^{-8}
X (two)	0.998051	174.9391°	X (0.999025)	0.999964	4.2×10^{-6}	1.8×10^{-8}

θ is the rotation angle of the even block; the excess over 90° (or 180°) is the hybridization phase $\int \varepsilon\, dt$ of the exchanged pair. "Full vs reduced" is the largest entry-wise difference of the even block (full amplitudes divided by the square root of the unexcited probability, global phase removed); the TN and dense (ED) reduced gates agree in $|M|$, P, θ and the leakage.

2 Method

2.1 Quasiparticle operators and the Majorana matrix

With the Nambu spinor Ψ and $H = \frac{1}{2}\Psi^\dagger H_{\mathrm{BdG}}\Psi$, the quasiparticle operator of a BdG vector w is $b(w) = w^\dagger\Psi$. It is *antilinear* in w: $b(c\,w) = c^* b(w)$. The Heisenberg evolution is $\hat{U}^\dagger\Psi\hat{U} = S_T\Psi$, with $S_T = \mathcal{T}\, e^{-i\int H_{\mathrm{BdG}}dt}$ the single-particle propagator, so

$$\hat{U}^\dagger b(w)\hat{U} = b(S_T^\dagger w), \qquad \hat{U} b(w)\hat{U}^\dagger = b(S_T w).$$

For self-conjugate Majoranas ($C\gamma = \gamma$, C the particle–hole conjugation) $\hat{\gamma}_j = b(\gamma_j)$ is Hermitian, and $\hat{U}\hat{\gamma}_j\hat{U}^\dagger = b(S_T\gamma_j) = \sum_i \hat{\gamma}_i M_{ij}$ with the real matrix $M_{ij} = \langle\gamma_i|S_T|\gamma_j\rangle$ (real because $[S_T, C] = 0$). Without leakage out of the zero-mode subspace M is orthogonal; $\|M^T M - 1\|$ measures the leakage.

2.2 Spin representation and the Jordan–Wigner convention

Restricted to the zero-mode Fock space, $\hat{U}$ acts as a unitary $\tilde{U}$ with $\tilde{U}\hat{\gamma}_j\tilde{U}^\dagger = \sum_i \hat{\gamma}_i M_{ij}$. With $M = e^A$ (A real antisymmetric), $\tilde{U} = \exp(\frac{1}{4}\sum_{ij} A_{ij}\hat{\gamma}_i\hat{\gamma}_j)$, up to a global phase and the sign of A (SO(4) is double covered); the sign is fixed by checking the defining relation.

The $\hat{\gamma}_i$ are represented on $|n_A n_B\rangle$ by Jordan–Wigner, with $f_A = a \otimes 1$, $f_B = Z \otimes a$. **The convention matters**: f_m must annihilate the positive-energy mode ψ_m, so that $n_m = 1$ means ψ_m occupied as in the full method. By antilinearity,

$$b(\psi_m) = b\Big(\tfrac{\gamma_{2m-1} + i\gamma_{2m}}{\sqrt{2}}\Big) = \frac{\hat{\gamma}_{2m-1} - i\hat{\gamma}_{2m}}{\sqrt{2}} \quad \Rightarrow \quad \hat{\gamma}_{2m-1} = f_m + f_m^\dagger, \qquad \hat{\gamma}_{2m} = +i(f_m - f_m^\dagger).$$

The opposite sign, $\hat{\gamma}_{2m} = -i(f_m - f_m^\dagger)$, used in the earlier `msh_logical_gate.jl` / `msh_pfaffian` work, makes f_m the creation operator of ψ_m: it swaps the labels $|00\rangle \leftrightarrow |11\rangle$ and conjugates every relative phase (one `exchange_12` then appears as $\mathrm{diag}(1, e^{+i\,90.72^\circ})$ instead of $\mathrm{diag}(1, e^{-i\,90.72^\circ})$). The Python notebook checks the convention against the full Pfaffian amplitudes entry by entry (section 4.6).

2.3 Full method: generalized Wick theorem

A Gaussian state is a set of $2N$ occupied columns of the frame ψ_0 (the fixed BdG unitary of `fix_unitary`: ψ_A, ψ_B in columns $2N$, $2N+1$, their partners $C\psi$ in $2N-1$, $2N-2$). For occupations k, $|k\rangle = a_{m_1}^\dagger a_{m_2}^\dagger \cdots |00\rangle$ ($m_1 < m_2$), where the creation operator of mode m is $b(C\phi_m)$ and its

adjoint $b(\phi_m)$, ϕ_m = column $2N + m$. Writing both states relative to a reference $|r\rangle$,

$$\frac{\langle j|\hat{U}|k\rangle}{\langle r|\hat{U}|r\rangle} = \langle r|X(r)\,X^\dagger(j)\,\hat{U}\,X(k)\,X^\dagger(r)|r\rangle_{\text{normalized}},$$

and moving the operators left of $\hat{U}$ through it with $S_T^\dagger$, the normalized expectation value of the operator string is the Pfaffian of the antisymmetric matrix of pairwise contractions

$$C_{ij} = x_i^\dagger(1-\rho)\,Cx_j, \qquad 1-\rho = 1 - W_r G^{-1} W_L^\dagger, \quad W_L = S_T^\dagger W_r, \quad G = W_L^\dagger W_r,$$

with W_r the occupied columns of $|r\rangle$. $1-\rho$ is the mixed (transition) density between $\langle r|$ and $\hat{U}|r\rangle$; this is where the whole occupied frame and its evolution enter. $|\langle r|\hat{U}|r\rangle| = |\det G|^{1/2}$; its sign is not fixed but common to all amplitudes, so moduli and relative phases are meaningful and the global phase is not. The probability of staying unexcited for input k is $\sum_j |\langle j|\hat{U}|k\rangle|^2$ over the logical outputs j (odd outputs vanish by parity).

3 Part I — `logical_gates_tn.ipynb` (Julia)

No time evolution: everything is 4×4 algebra on the overlaps cached by `msh_tjunctions_tn.ipynb`; the notebook runs in seconds.

3.1 Section 1: setup and the cached overlaps

`T1`

Duration of one exchange, as `duration` in `msh_tjunctions_tn.ipynb`: $T_1 = (0.1\,(4L_1 + 2L_2 + 8) + 1)\cdot 200 = 2040$ for `exchange_12` and $(0.1\,(L_1 + 3L_2 + 2L_1 + 3) + 1)\cdot 400 = 3880$ for `exchange_23` with $L_1 = L_2 = 14$.

`runs, overlaps(kind, t)`

`runs[kind]` is the deserialized cache `msh_tjunctions_tdvp_$kind.jls` (`kind` = `"Z"` or `"X"`); a missing cache stops the notebook with an instruction to run `msh_tjunctions_tn.ipynb` first. `overlaps(kind, t)` returns the 4×2 matrix $z_{ik} = \langle\gamma_i(0)|\psi_k(t)\rangle$ at the snapshot t (an error if there is none; snapshots are every 5 time units during the gate, so T_1 and $2T_1$ are included). Printed: $T_1 = 2040$ (Z, snapshots to $t = 4460$) and 3880 (X, to 8140).

3.2 Section 2: from the Majorana matrix to the logical gate

`braid_matrix(z)`

M from the overlaps. With $\psi_k = (\gamma_{2k-1} + i\gamma_{2k})/\sqrt{2}$, $C\gamma = \gamma$ and $[S_T, C] = 0$: $\gamma_{2k-1}(T) = (\psi_k(T) + C\psi_k(T))/\sqrt{2}$, $\gamma_{2k}(T) = (\psi_k(T) - C\psi_k(T))/(i\sqrt{2})$, and $\langle\gamma_i|C\psi\rangle = \langle\psi|\gamma_i\rangle = z_{ik}^*$, so

$$M_{i,2k-1} = \sqrt{2}\,\mathrm{Re}\,z_{ik}, \qquad M_{i,2k} = \sqrt{2}\,\mathrm{Im}\,z_{ik}.$$

Only $\psi_A(T)$ and $\psi_B(T)$ are needed: the TDVP never evolves the partners, which follow from C exactly.

`jw_majoranas()`

The four $\hat{\gamma}_i$ as 4×4 matrices on $|n_A n_B\rangle$, order $00,01,10,11$: $f_A = a\otimes 1$, $f_B = Z\otimes a$ with $a = |0\rangle\langle 1|$, $\hat{\gamma}_{2m-1} = f_m + f_m^\dagger$, $\hat{\gamma}_{2m} = +i(f_m - f_m^\dagger)$ (the convention of section 2.2).

`logical_unitary(M)`

Replaces M by its nearest orthogonal matrix $M_o = UV^T$ (SVD; this renormalizes away the leakage), requires $\det M_o > 0$ (a parity-preserving rotation), takes $A = \log M_o$ (antisymmetrized), and for both signs of A forms $\tilde{U} = \exp(\pm\frac{1}{4}\sum A_{ij}\hat{\gamma}_i\hat{\gamma}_j)$, keeping the one with the smaller residual $\max_j \|\tilde{U}\hat{\gamma}_j\tilde{U}^\dagger - \sum_i \hat{\gamma}_i M_{o,ij}\|$. Returns $(\tilde{U}, \text{residual})$; the residuals are $(1\text{–}3)\times10^{-15}$.

`IDEAL, logical_even_block(Ũ)`

`IDEAL` lists the ideal gates $\sqrt{X} = e^{-i\pi X/4}$, $\sqrt{X}^\dagger$, X, $\sqrt{Z} = \mathrm{diag}(1,i)$, $\sqrt{Z}^\dagger$, Z. `logical_even_block` takes the even block U_L of $\tilde{U}$ on $\{|00\rangle,|11\rangle\}$, normalizes $W = U_L/\sqrt{\det U_L}$ and reads off the rotation angle and axis from $W = \cos\frac{\theta}{2} - i\sin\frac{\theta}{2}\,\hat{n}\cdot\sigma$; it returns U_L, θ, $\hat{n}$, $P(00\to11) = |U_{L,21}|^2$ and the global-phase-insensitive overlaps $|\mathrm{Tr}(V^\dagger U_L)|/2$ with every ideal gate V.

Check. An ideal braid $\gamma_2\to\gamma_3$, $\gamma_3\to-\gamma_2$ gives residual 8×10^{-16}, $\theta = 90°$, $\hat{n} = (-1,0,0)$: exactly $\sqrt{X}^\dagger$ (overlap 1.000000).

3.3 Section 3: logical gates after one and two exchanges

For (Z,T_1), $(Z,2T_1)$, (X,T_1), $(X,2T_1)$: M from `braid_matrix(overlaps(...))`, the leakage $1-\sum_i|z_{ik}|^2$ of each mode, $\|M^TM-1\|$, the spin-representation residual, $P(00\to11)$, θ, $\hat{n}$ and the ideal-gate overlaps. Printed values:

Gate	t	leak (ψ_A,ψ_B)	$\|M^TM-1\|$	$P(00\to11)$	θ	$\hat{n}$
$\sqrt{Z}^\dagger$	2040	$(9.72\times10^{-7}, 1.65\times10^{-11})$	1.4×10^{-6}	0	90.7180°	$(0,0,-1)$
Z	4080	$(1.99\times10^{-6}, 2.07\times10^{-11})$	2.8×10^{-6}	0	178.5641°	$(0,0,1)$
$\sqrt{X}^\dagger$	3880	$(3.49\times10^{-5}, 1.23\times10^{-7})$	7.0×10^{-5}	0.522078	92.5308°	$(-1,0,0)$
X	7760	$(3.42\times10^{-5}, 3.49\times10^{-5})$	1.0×10^{-4}	0.998051	174.9391°	$(1,0,0)$

The ideal-gate overlaps are 0.999980 ($\sqrt{Z}^\dagger$), 0.999921 (Z), 0.999756 ($\sqrt{X}^\dagger$) and 0.999025 (X). One `exchange_12` leaves pair B untouched (its leakage stays at 10^{-11}); one `exchange_23` acts on both pairs.

3.4 Section 4: export

`logical_gates_tn.csv`: columns `gate, stage, t, leak_A, leak_B, M11 …M44` (row-major), one row per gate and stage.

4 Part II — `logical_gates_python_ed.ipynb` (Python)

4.1 Code from `tjunctions_model.py`

The notebook imports the model (`from tjunctions_model import *`); these functions are documented in Part II of the T-junction reference and only summarized here.

Name	Role here
`H0`, `ham(f)`, `ham_sp(f)`	H_{BdG} of the initial texture f_0; dense and sparse H for a texture $f(c, s, n)$
`Exchange12`, `Exchange23`, `f_array_12`, `f_array_23`	the two protocols (durations, textures)
`zero_mode_frame(H)`	`eigh`, `individual_MZMs` (Takagi + localization), `fix_unitary`: the fixed frame ψ_0 with ψ_A, ψ_B in columns $2N$, $2N+1$ and partners in $2N-1$, $2N-2$
`step(psi, t, dt, ft)`, `DT`	2-stage Gauss–Legendre step (sparse LU), $dt = 0.25$
`Cop(v)`	particle–hole conjugation of the kwant spinor, $C = \tau_y \sigma_y K$
`N`, `xs`	144 sites; x coordinate of each site

4.2 Section 1: setup

Cell 1 sets `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS` to 1 before NumPy is imported (single thread, as in `tjunctions_python_ed.ipynb`), imports the model and builds `psi0, E0, mbs, mask = zero_mode_frame(H0)`; `states` lists the occupations $(0,0), (0,1), (1,0), (1,1)$. Printed: $N = 144$, $D = 576$, $\|\psi_0^\dagger \psi_0 - 1\| = 6.5 \times 10^{-14}$, mode energies 1.09×10^{-9} and 1.16×10^{-9}.

4.3 Section 2: time evolution of the full frame

`T1, gate_f`

T_1 from `p12.duration()` and `p23.duration()` (`Exchange23(rot_T=400.0)`); `gate_f[kind](t)` is the texture of two consecutive exchanges, the second one starting at T_1.

`evolve_frame(ft, psi, t0, t1, dt=DT)`

Evolves the whole $4N \times 4N$ frame from t_0 to t_1 with `step`, renormalizing the columns after each step: `evolve` of `tjunctions_python_ed.ipynb` without the readout.

Cache. If `frames.npz` is missing or `RUN_EVOLUTION = True`, both gates are evolved and saved after one and two exchanges (`Z1, Z2, X1, X2`) together with ψ_0. When loading, the notebook uses the cached ψ_0 (the phases of degenerate bulk eigenvectors from `eigh` depend on the BLAS build and threads, so a recomputed ψ_0 need not match bit for bit) and checks that it is an eigenframe of H_0 whose zero-mode columns equal those of `zero_mode_frame(H0)`. The propagators are $S_T = \psi(T)\psi_0^\dagger$; printed $\|S_T^\dagger S_T - 1\| = 4 \times 10^{-12}$ to 1.3×10^{-11}. Evolution times of the cached run: Z 692 s, X 1363 s (single thread, measured while other jobs ran on the machine).

4.4 Section 3: full method

`pfaffian(A)`

Pfaffian of a complex antisymmetric matrix by Parlett–Reid elimination with pivoting: columns in pairs $(k, k+1)$; the largest entry of column k below the diagonal is swapped into row and column $k+1$ (each swap flips the sign); the pivot $A_{k,k+1}$ is a factor; the trailing block gets the antisymmetric rank-2 update $A \leftarrow A + \tau a^T - a\tau^T$, $\tau = A_{k,k+2:}/A_{k,k+1}$. Check: $|\mathrm{Pf}(A)^2 - \det A| = 3.7 \times 10^{-11}$ for a random 8×8 matrix.

`Cvec, mask_of(bits)`

`Cvec = Cop.` `mask_of` marks columns $2N + m$ with $n_m = 1$ and mirrors: column $2N - 1 - m$ is occupied exactly when $2N + m$ is empty, so every state occupies $2N$ columns (the CZ notebooks' `masks[:, :2N] = flip(~masks[:, 2N:])`).

`signed_amplitudes(pairs, S_T, psi0, candidates=states)`

$\langle j|\hat{U}|k\rangle$ for all (j,k) in `pairs`, up to one common sign (section 2.3). The reference r is the candidate with the largest $|\det(W_r^\dagger S_T W_r)|$; $W_L = S_T^\dagger W_r$, $G = W_L^\dagger W_r$, `norm_r` $= |\det G|^{1/2}$. For each pair the operator vectors are $S_T^\dagger X(r)$, $S_T^\dagger X^\dagger(j)$, $X(k)$, $X^\dagger(r)$ with $X(\text{bits}) = [C\phi_m]$ for occupied m (ascending) and $X^\dagger = [\phi_m]$ (descending); the contraction matrix is $P^\dagger CP - (P^\dagger W_r)G^{-1}(W_L^\dagger CP)$ (the $4N \times 4N$ mixed density is never formed), its strict upper triangle antisymmetrized, and the amplitude is its Pfaffian times `norm_r`. An empty string gives `norm_r`. Returns the amplitudes, r and `norm_r`. This is `projected_amplitudes` of `period_finding_signed_amplitudes.py` without mid-protocol projections.

Checks and references. With $S_T = 1$, $\max|A_{jk} - \delta_{jk}| = 3.9 \times 10^{-15}$. References chosen: $r = (0,1)$ with $|\langle r|\hat{U}|r\rangle| = 0.999999$ for Z1 and Z2, 0.691313 for X1, and $r = (0,0)$ with 0.044149 for X2 (an X gate empties the diagonal; the amplitudes are still consistent, see below).

4.5 Section 4: reduced method

`gamma0, braid_matrix(S_T)`

`gamma0` holds $\gamma_{2k-1} = (\psi_k + C\psi_k)/\sqrt{2}$ and $\gamma_{2k} = (\psi_k - C\psi_k)/(i\sqrt{2})$ from the frame (the same convention as the TN notebook); printed $\|\gamma^\dagger\gamma - 1\| = 2.3 \times 10^{-15}$, $\max\|C\gamma - \gamma\| = 0$, centres $x = -46.6, -17.5, 16.5, 45.6$. `braid_matrix` returns $\mathrm{Re}\,(\gamma^\dagger S_T \gamma)$ and $\max|\mathrm{Im}|$ ($\leq 2.3 \times 10^{-15}$).

`jw_majoranas(), logical_unitary(M), IDEAL, logical_even_block(U)`

The functions of Part I in Python (`scipy.linalg.logm`, `expm`, `svd`), with the same Jordan–Wigner convention. Residuals $\leq 2.4 \times 10^{-15}$; $\|M^T M - 1\|$ as in the TN notebook.

4.6 Section 5: full vs reduced

Per gate and stage: the even-block probabilities of the full method, the unexcited probability per input, the full probabilities renormalized by it, the reduced probabilities, the diagonal (and, for X-type gates, off-diagonal) relative phases of both methods, and the whole even block entry by entry: $\max|U_{\text{full}}/\sqrt{P_{\text{stay}}} - \tilde{U}|$ after removing the global phase (fixed on the largest entry).

Gate	unexcited prob.	$\arg\langle 11\|\hat{U}\|11\rangle - \arg\langle 00\|\hat{U}\|00\rangle$ full / reduced	$\arg\langle 11\|\hat{U}\|00\rangle - \arg\langle 00\|\hat{U}\|00\rangle$ full / reduced	even block entry by entry
$\sqrt{Z}^\dagger$	0.999998	$-90.7180°$ / $-90.7180°$	–	3.3×10^{-12}
Z	0.999997	$+178.5641°$ / $+178.5641°$	–	1.0×10^{-11}
$\sqrt{X}^\dagger$	0.999982	$-0.0002°$ / $-0.0002°$	$+89.9999°$ / $+89.9999°$	9.8×10^{-8}
X	0.999964	$+0.0050°$ / $+0.0050°$	$-89.9920°$ / $-89.9975°$	4.2×10^{-6}

The renormalized full probabilities equal the reduced ones to 3.5×10^{-29} (Z) and 1–3×10^{-11} (X). The remaining entry-wise differences of the X-type gates (10^{-7}–10^{-6}) are of the order of the bulk

excitation ($1 - P_{\text{stay}} = 1.8 \times 10^{-5}$ and 3.6×10^{-5}), which only the full method contains. With the opposite Jordan–Wigner sign the phases come out conjugated and the entry-wise difference is of order 1.

4.7 Section 6: comparison with the tensor-network gates

Reads `logical_gates_tn.csv`, rebuilds $\tilde{U}$ from M_{TN} with the Python `logical_unitary`, and compares quantities that do not depend on the sign of each γ (fixed independently in the two codes): $\max||M_{\text{TN}}| - |M_{\text{ED}}||$, $P(00 \to 11)$, θ and the leakage $1 - \sum_i |\langle\gamma_i|S_T\psi_k\rangle|^2$.

Gate	$\max\|\|M_{\text{TN}}\| - \|M_{\text{ED}}\|\|$	$P(00 \to 11)$ TN / ED	θ TN / ED	leak TN (A, B)	leak ED (A, B)
$\sqrt{Z}^\dagger$	4.9×10^{-9}	0 / 0	90.7180° / 90.7180°	$(9.72 \times 10^{-7}, 1.7 \times 10^{-11})$	$(9.74 \times 10^{-7}, 0)$
Z	8.5×10^{-9}	0 / 0	178.5641° / 178.5641°	$(1.99 \times 10^{-6}, 2.1 \times 10^{-11})$	$(2.00 \times 10^{-6}, 0)$
$\sqrt{X}^\dagger$	1.6×10^{-8}	0.522078 / 0.522078	92.5308° / 92.5308°	$(3.49 \times 10^{-5}, 1.23 \times 10^{-7})$	$(3.49 \times 10^{-5}, 1.24 \times 10^{-7})$
X	1.8×10^{-8}	0.998051 / 0.998051	174.9391° / 174.9391°	$(3.42 \times 10^{-5}, 3.49 \times 10^{-5})$	$(3.42 \times 10^{-5}, 3.49 \times 10^{-5})$

The MPS/TDVP evolution of the two mode vectors (bond dimension ≤ 64, $dt = 0.1$) and the dense Gauss–Legendre evolution of the full frame ($dt = 0.25$) give the same logical gates to $\sim 10^{-8}$; the TN route needs only the two vectors and no dense matrix.

5 `tjunctions_model.py`

The Python model of the array, moved out of `tjunctions_python_ed.ipynb` (whose outputs are unchanged by the move) so that both Python notebooks use one copy. Contents, in order: environment variables for single-thread BLAS (effective only if the module is imported before NumPy; the notebooks also set them in their first cell); Pauli matrices and their Kronecker products; geometry (`NLEG`, `NCELL`, `center_x`, `array_pos`, `label`, `build_system_3T`, and the module-level `sys`, `N`, `tags`, `xs`, `ys`); textures and protocols (`smoothstep`, `f0`, `spins`, `Exchange12`, `Exchange23`, `Fusion` and their texture functions); the Hamiltonian (`params`, `ham_kwant`, `H_noJ`, `exchange_matrix`, `ham`, `Cop`, `H0`); the zero-mode construction (`dag`, `PHconj`, `takagi`, `individual_MZMs`, `fix_unitary`); two functions added with the move, `zero_mode_frame(H)` (returns `psi0, E, mbs, mask`) and `logical_masks()` (the $4 \times 4N$ masks of $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$); the sparse Hamiltonian and Gauss–Legendre step (`exchange_sparse`, `ham_sp`, `step`, `DT`); and `electron_density(W)`. All are described in Part II of `docs/msh_tjunctions_function_reference.pdf`.